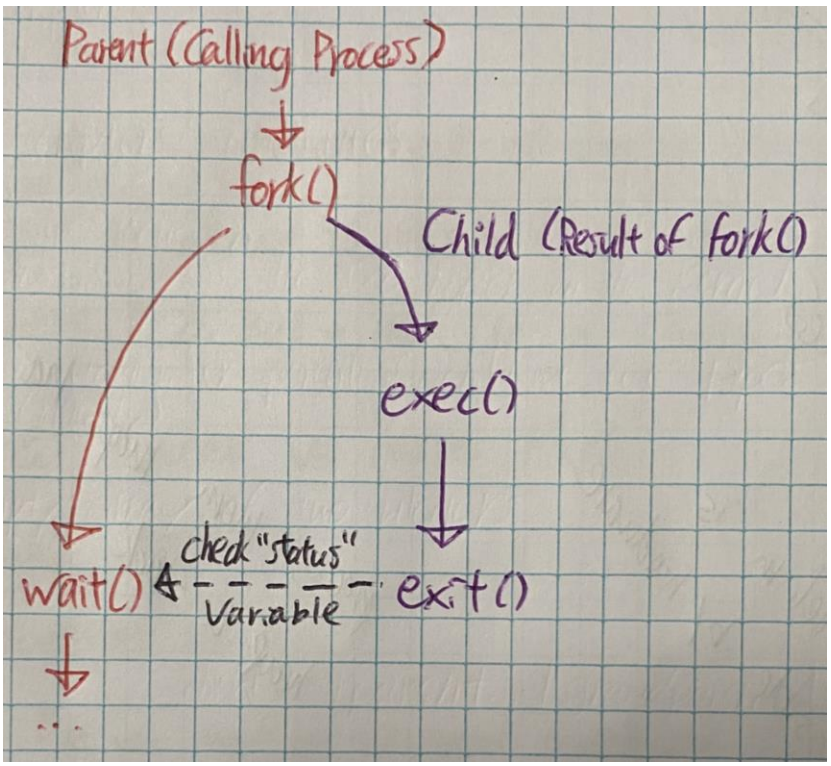
Design (based on provided comments in the shell template)

- If the user's typed string ends with **`&`** or **`;`** replace those characters with a **`blankspace`**.
- Use the C++ string library to tokenize the input into an array.
- Because of the way **`execvp()`** executes, we NULL-terminate the array.
- Check for **`exit`** and **`fg`**, which are commands that do *not* require the usage of **`execvp()`**. They can be implemented without spawning any child processes because:
  - **`exit`** ends the entire command interpreter.
  - **`fg`** brings an existing background child to the foreground.
- To implement **`fg`**, use Linux's **`waitpid()`** system call to suspend the current process. After checking the return **`&status`** of the topmost process in the background stack and ensuring it hasn't exited yet, immediately give that process the CPU.

Heavy lifting of shell.cpp begins at this point



- **`fork()`** is called once but returns twice. The child process isn't an exact copy because its process ID is different.
- Personally, I dislike using the variable name **`pid`** to store the fork's return value. This isn't the actual ID number the kernel sees in the Process Control Block.
- In our program, **`pid`** is not meant to be an accurate reflection of the PCB. We only care if **`pid`** is 0 or larger. This controls the **`if-else`** statement within parent and child.
- The child with **`pid`** 0 simply calls **`execvp()`** to find an executable file for the first token in the keyboard input. There is no other code written in this block because **`execvp()`** replaces the child shell with the file it chose to execute.
  - We don't call **`execvp()`** in the parent because it would consume the calling process, which would consume the command interpreter, and make the program useless for the human user.

- The parent waits in its own line of execution. It does the same **`&status`** check as **`fg`**.
- Deterministic execution is ensured because, like a good in-real-life parent, the calling process won't continue until its child is finished.

Testing and Output

I used the provided a.out executable file as the baseline to compare to. I couldn't use the Linux OS proper for testing because it wouldn't allow me to test **`exit`**. Linux also gives actual process IDs when **`sleep`** is called.

My implementation and a.out are very limited. When a valid command is immediately followed by an invalid one (i.e., pwd; p) the program doesn't crash, but also doesn't give an error message. It simply prints the $ prompt again. If I type two invalid commands (i.e., as gas), I have to manually kill the current process with the keyboard shortcut.

Extremely basic commands like pwd; echo; man work as expected. However, after calling **`ls`** I must call **`exit`** multiple times.

Output

My Implementation

```
[hreyes@csslab9 Final Boss]$ ./my_shell.out
$ ps -u hreyes;
    PID TTY          TIME CMD
 438387 ?        00:00:00 systemd
 438388 ?        00:00:00 (sd-pam)
 438397 ?        00:00:00 sshd
 438398 pts/6    00:00:00 bash
 438452 pts/6    00:00:00 my_shell.out
 438455 pts/6    00:00:00 ps
$ who
hreyes   pts/6        2026-04-16 21:19 (10.102.17.229)
$ sleep 40&
$ sleep 20&
$ fg
$ fg
$ exit
[hreyes@csslab9 Final Boss]$ |
```

a.out Executable

```
[hreyes@csslab9 Final Boss]$ ./a.out
$ ps -u hreyes;
    PID TTY          TIME CMD
 438387 ?        00:00:00 systemd
 438388 ?        00:00:00 (sd-pam)
 438397 ?        00:00:00 sshd
 438398 pts/6    00:00:00 bash
 438509 pts/6    00:00:00 a.out
 438514 pts/6    00:00:00 ps
$ who
hreyes   pts/6        2026-04-16 21:19 (10.102.17.229)
$ sleep 40&
$ sleep 20&
$ fg
$ exit
[hreyes@csslab9 Final Boss]$ |
```

References

- https://www.ibm.com/docs/en/zvm/7.3.0?topic=descriptions-waitpid-wait-specific-child-process-end
- https://pages.cs.wisc.edu/~remzi/OSTEP/
- Parsons, Erika. "Lecture Slides." CSS 430: Operating Systems, Microsoft OneNote, 2026, https://onedrive.live.com/personal/14b3ef2d44442c5a/_layouts/15/Doc.aspx?sourcedoc=%7B44442c5a-ef2d-20b3-8014-0c0400000000%7D
- Operating Systems 10th Edition